



622. Design Circular Queue

Description:

- ✓ **Design your implementation of the circular queue.**
- ✓ The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".
- ✓ **Implement the MyCircularQueue class:**
 - 1) **MyCircularQueue(k)**: Initializes the object with the size of the queue to be k.
 - 2) **int Front()**: Gets the front item from the queue. If the queue is empty, return -1.
 - 3) **int Rear()**: Gets the last item from the queue. If the queue is empty, return -1.
 - 4) **boolean enQueue(int value)**: Inserts an element into the circular queue. Return true if the operation is successful.
 - 5) **boolean deQueue()**: Deletes an element from the circular queue. Return true if the operation is successful.
 - 6) **boolean isEmpty()**: Checks whether the circular queue is empty or not.
 - 7) **boolean isFull()**: Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Constraints:

- a) $1 \leq k \leq 1000$
- b) $0 \leq \text{value} \leq 1000$
- c) At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 3

Amazon 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft 13

Facebook 8

Nvidia 6

Airbnb 4

Intuit 4

Rubrik 3

Google 2

622. Design Circular Queue

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
* */
```

```
class MyCircularQueue {
```

```
    private int capacity; // size of array
```

```
    private int size; // size of the Queue
```

```
    private int front;
```

```
    private Integer myarray[];
```



```
public MyCircularQueue(int capacity) {
    this.capacity = capacity;
    this.size = 0;
    this.front = 0;
    this.myarray = new Integer[capacity];
}

public boolean enqueue(int element) {
    if (size == capacity) {
        return false;
    }

    myarray[(front + size) % capacity] = element;
    size++;
    return true;
}

public boolean dequeue() {
    if (size == 0) {
        return false;
    }
    myarray[front] = null;
    front = (front + 1) % capacity;
    size--;
    return true;
}

public int Front() {
    if (size == 0)
        return -1;

    return myarray[front];
}
```



```
public int Rear() {  
    if (size == 0)  
        return -1;  
    int rear = (front + size - 1) % capacity;  
    return myarray[rear];  
}  
  
public int size() {  
    return size;  
}  
  
public boolean isEmpty() {  
    return (size == 0);  
}  
  
public boolean isFull() {  
    return (size == capacity);  
}  
}
```



641. Design Circular Deque

Description:

- ✓ Design your implementation of the circular double-ended queue (deque).
- ✓ Implement the MyCircularDeque class:
 - 1) **MyCircularDeque(int k)**: Initializes the deque with a maximum size of k.
 - 2) **boolean insertFront()**: Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
 - 3) **boolean insertLast()**: Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
 - 4) **boolean deleteFront()**: Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.
 - 5) **boolean deleteLast()**: Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
 - 6) **int getFront()**: Returns the front item from the Deque. Returns -1 if the deque is empty.
 - 7) **int getRear()**: Returns the last item from Deque. Returns -1 if the deque is empty.
 - 8) **boolean isEmpty()**: Returns true if the deque is empty, or false otherwise.
 - 9) **boolean isFull()**: Returns true if the deque is full, or false otherwise.

Constraints:

- a) $1 \leq k \leq 1000$
- b) $0 \leq \text{value} \leq 1000$
- c) At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.



Companies

1 year - 2 years

Goldman Sachs **2**

641. Design Circular Deque

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyCircularDeque {
    private int front;
    private int rear;
    private int size;
    private int capacity;
    private Integer myarray[];

    public MyCircularDeque(int capacity) {
        this.capacity = capacity;
        myarray = new Integer[capacity];
        this.front = -1;
        this.rear = 0;
        this.size = 0;
    }

    public boolean isFull() {
        return (size == capacity);
    }

    public int size() {
        return size;
    }
}
```



```
public boolean isEmpty() {
    return (size == 0);
}

public int getFront() {
    if (isEmpty()) {
        return -1;
    }
    return myarray[front];
}

public int getRear() {
    if (isEmpty()) {
        return -1;
    }
    return myarray[rear];
}

public boolean insertFront(int element) {
    if (isFull()) {
        return false;
    }

    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (front == 0) {
        front = capacity - 1;
    } else {
        front = front - 1;
    }

    myarray[front] = element;
    size++;
    return true;
}
```



```
public boolean insertLast(int element) {
    if (isFull())
        return false;

    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (rear == capacity-1) {
        rear = 0;
    } else {
        rear = rear + 1;
    }

    myarray[rear] = element;
    size++;
    return true;
}

public boolean deleteFront() {
    if (isEmpty())
        return false;

    myarray[front] = null;
    size--;

    if (front == rear) {
        front = -1;
        rear = -1;
    } else if (front == capacity-1) {
        front = 0 ;
    } else {
        front = front+1;
    }
    return true;
}
```



```
public boolean deleteLast() {  
    if (isEmpty()) {  
        return false;  
    }  
  
    myarray[rear] = null;  
    size--;  
  
    if (front == rear) {  
        front = -1;  
        rear = -1;  
    } else if (rear == 0) {  
        rear = capacity - 1;  
    } else {  
        rear = rear - 1;  
    }  
  
    return true;  
}  
}
```



225. Implement Stack using Queues

Description:

- ✓ Implement a last-in-first-out (LIFO) stack using only two queues.
- ✓ The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- 1) **void push(int x):** Pushes element x to the top of the stack.
- 2) **int pop():** Removes the element on the top of the stack and returns it.
- 3) **int top():** Returns the element on the top of the stack.
- 4) **boolean empty():** Returns true if the stack is empty, false otherwise.

Notes:

- ✓ You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.

Constraints:

- a) $1 \leq x \leq 9$
- b) At most 100 calls will be made to push, pop, top, and empty.
- c) All the calls to pop and top are valid.

Companies

6 months - 1 year

1 year - 2 years

Amazon **2**

Google **2**



Companies

6 months - 1 year

1 year - 2 years

Bloomberg 4

PayPal 3

Microsoft 2

225. Implement Stack using Queues

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyStack {

    Queue<Integer> myque1;
    Queue<Integer> myque2;

    public MyStack() {
        myque1 = new LinkedList<>();
        myque2 = new LinkedList<>();
    }

    public void push(int element) {

        //Step1: Add Element to myque2

        myque2.offer(element);

        //Step2: Move Elements from myque1 to myque2

        while(!myque1.isEmpty()) {
            myque2.offer(myque1.poll());
        }
    }
}
```



//Step3: Swap

```
        Queue<Integer> temp = myque1;
        myque1= myque2;
        myque2=temp;

    }

    public int pop() {
        return myque1.poll();
    }

    public int top() {
        return myque1.peek();
    }

    public boolean empty() {
        return myque1.isEmpty();
    }
}
```



232. Implement Queue using Stacks

Description:

- ✓ Implement a first in first out (FIFO) queue using only two stacks.
- ✓ The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- 1) **void push(int x):** Pushes element x to the back of the queue.
- 2) **int pop():** Removes the element from the front of the queue and returns that element
- 3) **int peek():** Returns the element at the front of the queue.
- 4) **boolean empty():** Returns true if the queue is empty, false otherwise.

Notes:

- ✓ You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.

Constraints:

- a) $1 \leq x \leq 9$
- b) At most 100 calls will be made to push, pop, peek, and empty.
- c) All the calls to pop and peek are valid.

Companies

6 months - 1 year

1 year - 2 years

Amazon 3

Microsoft 2

Apple 2



Companies

6 months - 1 year

1 year - 2 years

Goldman Sachs 5

VMware 3

eBay 2

ByteDance 2

Docusign 2

Bloomberg 2

Google 2

Oracle 2

232. Implement Queue using Stacks

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyQueue {

    Stack<Integer> mystack1;
    Stack<Integer> mystack2;

    public MyQueue() {
        mystack1 = new Stack<>();
        mystack2 = new Stack<>();
    }

    public void push(int element) {
        mystack1.push(element);
    }

    public int pop() {

        if (mystack2.isEmpty()) {
            while (!mystack1.isEmpty()) {
                mystack2.push(mystack1.pop());
            }
        }
    }
}
```



```
        int popElement = mystack2.pop();

        while (!mystack2.isEmpty()) {
            mystack1.push(mystack2.pop());
        }

        return popElement;
    }

    public int peek() {
        if (mystack2.isEmpty()) {
            while (!mystack1.isEmpty()) {
                mystack2.push(mystack1.pop());
            }
        }
        int topElement = mystack2.peek();

        while (!mystack2.isEmpty()) {
            mystack1.push(mystack2.pop());
        }

        return topElement;
    }

    public boolean empty() {
        return mystack1.isEmpty();
    }
}
```



239. Sliding Window Maximum

Description:

- ✓ You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right.
- ✓ You can only see the k numbers in the window.
- ✓ Each time the sliding window moves right by one position.
- ✓ Return the max sliding window.

Example 1:

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [3,3,5,5,6,7]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Constraints:

- a) $1 \leq \text{nums.length} \leq 105$
- b) $-104 \leq \text{nums}[i] \leq 104$
- c) $1 \leq k \leq \text{nums.length}$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 4

Yandex 3

Uber 2

Microsoft 2

Google 2

Adobe 2



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 5

DoorDash 4

Citadel 3

Facebook 2

Bloomberg 2

HRT 2

TikTok 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Goldman Sachs 9

Salesforce 9

Coinbase 8

ByteDance 6

Twitter 5

Twilio 5

VMware 4

Booking.com 4

DE Shaw 4

Rubrik 3

Walmart Labs 2

Akuna Capital 2

Atlassian 2

TuSimple 2

Quora 2

Cruise Automation 2

Capgemini 2

Zenefits

239. Sliding Window Maximum

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {

        Deque<Integer> mydeque = new ArrayDeque<>();

        int n = nums.length;
        int resultArr[] = new int[n - k + 1];

        if (n * k == 0) {
            return new int[0];
        }
    }
}
```



```
if (k == 1) {
    return nums;
}

for(int i=0;i<k;i++) {

    while(!mydeque.isEmpty() && nums[i]>=nums[mydeque.peekLast()]) {
        mydeque.removeLast();
    }

    mydeque.addLast(i);
}

resultArr[0] = nums[mydeque.peek()];

for(int i=k;i<n;i++) {

    if(!mydeque.isEmpty() && mydeque.peek()==i-k) {
        mydeque.removeFirst();
    }

    while(!mydeque.isEmpty() && nums[i]>=nums[mydeque.peekLast()]) {
        mydeque.removeLast();
    }

    mydeque.addLast(i);
    resultArr[i-k+1] = nums[mydeque.peek()];
}

return resultArr;
}
```